

计算机系统设计PA4实验报告

苏耀磊 2311727

一、实验目的和环境概述

- 掌握在 NEMU 中实现时钟中断与中断响应的基本方法，理解硬件中断从设备触发、CPU 响应到内核处理的完整链路。
- 理解 eflags.IF、挂起中断标志与中断门机制之间的关系，学会通过中断屏蔽位控制用户态程序的可抢占执行。
- 掌握在 AM / Nanos-lite 中构造用户进程初始现场的方法，理解 _umake()、_trap() 与上下文切换之间的协作关系。
- 理解基于自陷和时钟中断的进程调度机制，学会通过调度器实现多个用户程序的分时运行与优先级控制。
- 通过实现用户程序的分时多任务运行，进一步理解操作系统中进程管理、事件驱动和中断机制的整体设计。
- 最终能够在系统上稳定运行仙剑奇侠传和 hello 程序，并观察到二者在时钟中断驱动下的分时切换效果。

本次实验环境使用 vscode 连接 docker 容器，在 debian 中进行实验，实验指导书参考 ics2017。

二、实验内容

- 计算机系统设计PA4实验报告
 - 一、实验目的和环境概述
 - 二、实验内容
 - 三、实验过程
 - 1.在NEMU中实现分页机制
 - 一些问题
 - 问题：空指针真的是"空"的吗？
 - 2.让用户程序运行在分页机制上
 - 问题：内核映射的作用
 - 3.在分页机制上运行仙剑奇侠传
 - 4.实现内核自陷
 - 5.实现上下文切换
 - 6.分时运行仙剑奇侠传和hello程序
 - 7.优先级调度
 - 8.添加时钟中断
 - 问题：灾难性的后果
 - 9.展示你的计算机系统
 - 四、必答题
 - 五、实验过程中的问题与解决方法

三、实验过程

1.在NEMU中实现分页机制

首先我们需要打开位于 nanos-lite/src/main.c 里的宏 HAS_PTE，然后 Nanos-lite 在初始化的时候首先就会调用 init_mm() 函数来初始化 MM。

而初始化 MM 的工作有两项，第一项工作是将 TRM 提供的堆区起始地址作为空闲物理页的首地址，将来会通过 new_page() 函数来分配空闲的物理页，第二项工作是调用 AM 的 _pte_init() 函数，填写内核的页目录和页表，然后设置 CR3 寄存器，最后通过设置 CR0 寄存器来开启分页机制。

因此在这里，我们需要先添加 CR3 寄存器和 CR0 寄存器，在 nemu/include/cpu/reg.h 里：

```
60 CR0 cr0;
61 CR3 cr3;
```

并在 restart() 函数中对 CR0 寄存器初始化为 0x60000011，在 nemu/src/monitor/monitor.c 里：

```
87 cpu.cr0.val = 0x60000011;
88 cpu.cr3.val = 0;
```

当然还有相应的操作它们的指令，make_DHelper(mov_r2cr) 和 make_DHelper(mov_cr2r)，在 nemu/src/cpu/decode/decode.c 里：

```
316 make_DHelper(mov_r2cr) {
317     decode_op_rm(eip, id_src, true, id_dest, false);
318     assert(id_src->type == OP_TYPE_REG);
319     assert(id_dest->type == OP_TYPE_REG);
320 }
321
322 make_DHelper(mov_cr2r) {
323     decode_op_rm(eip, id_dest, false, id_src, false);
324     assert(id_src->type == OP_TYPE_REG);
325     assert(id_dest->type == OP_TYPE_REG);
326 }
```

两个函数都是 x86 控制寄存器相关指令的译码辅助函数，在 CPU 执行指令之前，对当前指令中的操作数进行解析，并将解析结果保存到对应的操作数结构中，为后续执行阶段提供信息。

并在 nemu/include/cpu/decode.h 中补充函数声明：

```
115 make_DHelper(mov_r2cr);
116 make_DHelper(mov_cr2r);
```

还有位于 nemu/src/cpu/exec/system.c 的 make_EHelper(mov_cr2r) 和 make_EHelper(mov_r2cr) 函数：

```
18 make_EHelper(mov_r2cr) {
19     switch (id_dest->reg) {
20         case 0: cpu.cr0.val = id_src->val; break;
21         case 3: cpu.cr3.val = id_src->val; break;
22         default: panic("unsupported cr%d", id_dest->reg);
23     }
24
25     print_asm("movl %s,%crd", reg_name(id_src->reg, 4), id_dest->reg);
26 }
```

```

28  ∨ make_EHelper(mov_cr2r) {
29  ∨    switch (id_src->reg) {
30      case 0: id_dest->val = cpu.cr0.val; break;
31      case 3: id_dest->val = cpu.cr3.val; break;
32      default: panic("unsupported cr%d", id_src->reg);
33    }
34
35    operand_write(id_dest, &id_dest->val);
36
37    print_asm("movl %%cr%d,%%%s", id_src->reg, reg_name(id_dest->reg, 4));
38
39  ∨ #ifdef DIFF_TEST
40    diff_test_skip_qemu();
41  #endif
42  }

```

mov_r2cr 实现从普通寄存器向控制寄存器写入数据，也就是 mov cr0, eax 这样的指令。函数首先通过 switch(id_dest->reg) 判断目标控制寄存器是哪一个。

当 id_dest->reg 为 0 时，说明目标寄存器是 cr0，于是执行 cpu.cr0.val = id_src->val，将源操作数 id_src 中的值写入 cpu.cr0.val。

当 id_dest->reg 为 3 时，说明目标寄存器是 cr3，执行 cpu.cr3.val = id_src->val，把普通寄存器中的值写入寄存器 cr3。

如果目标控制寄存器既不是 cr0 也不是 cr3，就会调用 panic("unsupported cr%d", id_dest->reg)。

而 mov_cr2r 正好是相反的操作，不同的是，这里在修改 id_dest->val 后，还需要调用 operand_write(id_dest, &id_dest->val)来把数据真正写入目标寄存器。

然后在 nemu/src/cpu/exec/exec.c 中补充如下内容：

```

151  /* 0x1c */ EMPTY, EMPTY, EMPTY, EMPTY,
152  /* 0x20 */ IDEXW(mov_cr2r, mov_cr2r, 4), EMPTY, IDEXW(mov_r2cr, mov_r2cr, 4), EMPTY,
153  /* 0x24 */ EMPTY, EMPTY, EMPTY, EMPTY,

```

并在 nemu/src/cpu/exec/all-instr.h 中补充函数声明：

```

51  make_EHelper(mov_r2cr);
52  make_EHelper(mov_cr2r);

```

实现了这些以后，我们就可以对 vaddr_read() 和 vaddr_write() 函数作修改了，先补充一个 page_translate() 函数，在 nemu/src/memory/memory.c 中：

```

33 static paddr_t page_translate(vaddr_t addr, bool is_write) {
34     assert(cpu.cr0.protect_enable && cpu.cr0.paging);
35
36     uint32_t pde_index = (addr >> 22) & 0x3ff;
37     uint32_t pte_index = (addr >> 12) & 0x3ff;
38     uint32_t offset = addr & PAGE_MASK;
39
40     paddr_t pde_addr = (cpu.cr3.page_directory_base << 12) + pde_index * sizeof(PDE);
41     PDE pde;
42     pde.val = paddr_read(pde_addr, 4);
43     assert(pde.present);
44     if (!pde.accessed) {
45         pde.accessed = 1;
46         paddr_write(pde_addr, 4, pde.val);
47     }
48
49     paddr_t pte_addr = (pde.page_frame << 12) + pte_index * sizeof(PTE);
50     PTE pte;
51     pte.val = paddr_read(pte_addr, 4);
52     assert(pte.present);
53     if (!pte.accessed || (is_write && !pte.dirty)) {
54         pte.accessed = 1;
55         if (is_write) {
56             pte.dirty = 1;
57         }
58         paddr_write(pte_addr, 4, pte.val);
59     }
60
61     return (pte.page_frame << 12) | offset;
62 }

```

这个函数用来实现虚拟地址到物理地址的转换，输入虚拟地址 `addr` 和一个表示当前是否为写操作的参数 `is_write`，最终返回对应的物理地址，完整模拟分页机制中的页目录、页表查找过程，以及硬件自动维护访问位和脏位的行为。

之后就是 `vaddr_read` 与 `vaddr_write`：

```

64 uint32_t vaddr_read(vaddr_t addr, int len) {
65     if ((addr & PAGE_MASK) + len > PAGE_SIZE) {
66         int low_len = PAGE_SIZE - (addr & PAGE_MASK);
67         uint32_t low = vaddr_read(addr, low_len);
68         uint32_t high = vaddr_read(addr + low_len, len - low_len);
69         return low | (high << (low_len * 8));
70     }
71     else {
72         paddr_t paddr = addr;
73         if (cpu.cr0.protect_enable && cpu.cr0.paging) {
74             paddr = page_translate(addr, false);
75         }
76         return paddr_read(paddr, len);
77     }
78 }

```

`vaddr_read` 函数的从虚拟地址空间中读取数据，首先检查当前读取操作是否跨页。

如果发生跨页读取，函数会将读取操作拆成两部分。首先读取当前页剩余的数据长度 `low_len`，然后再读取下一页中的剩余部分。两次读取分别保存在 `low` 和 `high` 中，最后通过位运算将两部分数据拼接成完整结果返回。

如果读取没有跨页，先将虚拟地址 `addr` 暂时赋值给物理地址变量 `paddr`。随后程序检查当前 CPU 是否开启了保护模式和分页机制。如果分页机制已经开启，那么就调用 `page_translate(addr, false)` 将虚拟地址转换成真正的物理地址。

在得到物理地址之后，函数最终调用 `paddr_read(paddr, len)` 从物理内存中读取指定长度的数据，并将读取结果返回。

```
80 void vaddr_write(vaddr_t addr, int len, uint32_t data) {
81     if ((addr & PAGE_MASK) + len > PAGE_SIZE) {
82         int low_len = PAGE_SIZE - (addr & PAGE_MASK);
83         vaddr_write(addr, low_len, data & (~0u >> ((4 - low_len) << 3)));
84         vaddr_write(addr + low_len, len - low_len, data >> (low_len * 8));
85     }
86     else {
87         paddr_t paddr = addr;
88         if (cpu.cr0.protect_enable && cpu.cr0.paging) {
89             paddr = page_translate(addr, true);
90         }
91         paddr_write(paddr, len, data);
92     }
93 }
```

`vaddr_write` 函数向虚拟地址空间写入数据，首先检查当前写操作是否跨页。

如果发生跨页写入，首先计算当前页还能写入多少字节，并保存到 `low_len` 中。随后，程序先向当前页写入低地址部分的数据，再向下一页写入剩余的数据。

如果当前写入没有跨页，那么函数会先将虚拟地址 `addr` 临时赋值给物理地址变量 `paddr`。随后程序检查当前 CPU 是否已经开启保护模式和分页机制。如果分页已经开启，那么函数会调用 `page_translate(addr, true)` 将虚拟地址转换成物理地址。

在获得物理地址之后，函数最终调用 `paddr_write(paddr, len, data)`，将数据真正写入物理内存。

运行 `dummy` 程序，结果如下：

```
SuYaolei@988d7f56373a:~/code/ics2017/nanos-lite$ make run
./build/nemu -l /home/SuYaolei/code/ics2017/nanos-lite/build/nemu-log.txt /home/SuYaolei/code/ics2017/nanos-lite/build/nanos-lite-x86
[src/monitor/monitor.c,65,load_img] The image is /home/SuYaolei/code/ics2017/nanos-lite/build/nanos-lite-x86-nemu.bin
Welcome to NEMU!
[src/monitor/monitor.c,30,welcome] Build time: 14:08:50, May 21 2026
For help, type "help"
(nemu) c
[src/mm.c,44,init_mm] free physical pages starting from 0x1d97000
[src/main.c,20,main] 'Hello World!' from Nanos-lite
[src/main.c,21,main] Build time: 19:02:16, May 22 2026
[src/ramdisk.c,26,init_ramdisk] ramdisk info: start = 0x102bc0, end = 0x1d51805, size = 29682757 bytes
[src/main.c,28,main] Initializing interrupt/exception handler...
nemu: HIT GOOD TRAP at eip = 0x00100032
(nemu) █
```

实现是正确的。

一些问题

i386不是一个32位的处理器吗，为什么表项中的基地址信息只有20位，而不是32位？

虽然 i386 是 32 位处理器，但页目录项（PDE）、页表项（PTE）以及 CR3 中保存的页框基地址只有 20 位，这是因为页是按 4KB 对齐的，4KB 需要 12 位表示，因此一个页的起始地址的低 12 位一定是 0，既然低 12 位恒为 0，那么在页表项中就没有必要再存储它们，只需要存储高 20 位即可。当真正需要得到完整物理地址时，再通过左移 12 位恢复出完整的 32 位地址。

手册上提到表项(包括CR3)中的基地址都是物理地址，物理地址是必须的吗？能否使用虚拟地址？

页表项和 CR3 中必须保存物理地址，而不能使用虚拟地址。这是因为分页机制本身就是依靠页表来完成虚拟地址到物理地址转换的，如果页表里面再保存虚拟地址，那么 CPU 在访问页表时，又需要再次进行地址翻译，这样就会形成无限递归，系统将无法开始第一次地址转换。因此，页表和页目录必须位于物理内存中，并且 CR3 必须直接保存页目录的物理地址，这样 CPU 才能够绕过分页机制，直接访问页表结构。

为什么不采用一级页表？或者说采用一级页表会有什么缺点？

一级页表会导致巨大的内存浪费。在 32 位系统中，虚拟地址空间大小为 4GB。如果页面大小为 4KB，那么整个地址空间一共需要 2 的 20 次方个页表项，也就是大约 104 万个页表项。如果每个页表项占 4 字节，那么一个完整的一级页表就需要 4MB 内存。

问题在于，每个进程都必须拥有自己的页表。如果系统中同时运行很多进程，那么仅页表本身就会消耗大量内存。而实际上，大多数进程并不会使用完整的 4GB 虚拟地址空间，很多地址区域根本没有映射。如果使用一级页表，即使只使用了很小一部分虚拟地址空间，也必须提前分配完整的 4MB 页表，这会造成严重浪费。

两级页表的就可以按需分配，页目录只负责管理页表，而页表只有在对应虚拟地址区域真正被使用时才会创建。例如某个进程只使用了低端几 MB 内存，那么系统只需要创建少量页表即可，而不需要一次性分配整个 4MB 的一级页表。这样能够大幅降低页表带来的内存开销。

问题：空指针真的是“空”的吗？

程序设计课上老师告诉你，当一个指针变量的值等于 NULL 时，代表空，不指向任何东西。仔细想想，真的是这样吗？当程序对空指针解引用的时候，计算机内部具体都做了些什么？你对空指针的本质有什么新的认识？

空指针其实并不是没有指向任何东西，它本质上仍然是一个地址，只不过这个地址的值为 0。

在 C 语言中，NULL 通常就是整数 0。因此，当定义一个空指针时，本质上只是让这个指针变量保存了地址 0。例如一个指针变量被赋值为 NULL 后，CPU 并不会认为它是特殊对象，而只是把它当作一个普通的数值。对于处理器来说，指针本质上只是一个地址数字。

当程序对空指针进行解引用时，例如访问 *p，CPU 实际做的事情是把指针变量中的值当作内存地址，然后尝试访问该地址中的数据。如果此时 p 的值是 0，那么 CPU 就会尝试访问虚拟地址 0。

随后，内存管理单元会对这个虚拟地址进行地址翻译。CPU 会根据当前页表查找虚拟地址 0 是否存在合法映射。如果操作系统没有为地址 0 建立映射，那么对应页表项中的 Present 位就是 0，于是 CPU 会触发缺页异常，也就是 Page Fault。

操作系统接收到这个异常后，会发现程序正在访问一个非法地址，于是终止程序运行。在 Linux 中，这通常表现为 Segmentation Fault。因此，所谓空指针不能访问，并不是因为 CPU 能识别 NULL，也不是因为地址 0 天生不可访问，而是因为操作系统故意不映射这个地址，从而利用硬件异常机制来捕获非法访问。

因此，指针本质上只是地址，而所谓“空指针”，只是一个被操作系统特殊保护起来的地址值。

2. 让用户程序运行在分页机制上

根据指导书内容，首先需要将 navy-apps/Makefile.compile 中的链接地址 -Ttext 参数改为 0x8048000，避免用户程序的虚拟地址空间与内核相互重叠：

```

21 ifeq ($(LINK), dynamic)
22     CFLAGS += -fPIE
23     CXXFLAGS += -fPIE
24     LDFLAGS += -fpie -shared
25 else
26     LDFLAGS += -Ttext 0x8048000
27 endif

```

同样的，nanos-lite/src/loader.c 中的 DEFAULT_ENTRY 也需要作相应的修改：

```

5 #define DEFAULT_ENTRY ((void *)0x8048000)

```

然后，我们让 Nanos-lite 通过 load_prog() 函数来进行用户程序的加载，在 nanos-lite/src/main.c 里：

```

34 // uint32_t entry = loader(NULL, "/bin/dummy");
35 // ((void (*)(void))entry)();
36
37 load_prog("/bin/dummy");

```

然后我们需要知道的是 load_prog() 会调用 loader() 函数加载用户程序，loader() 要做的事情是，获取用户程序的大小之后，以页为单位进行加载：

- 申请一页空闲的物理页
- 把这一物理页映射到用户程序的虚拟地址空间中
- 从文件中读入一页的内容到这一物理页上

用户进程在将来使用虚拟地址访问内存，在 loader 为用户进程准备的映射下，虚拟地址被转换成物理地址，通过这一物理地址访问到的物理内存，为了提供映射一页的功能，需要实现 _map() 函数，在 nexus-am/am/arch/x86-nemu/src/pte.c 里：

```

69 void _map(_Protect *p, void *va, void *pa) {
70     uint32_t pdir_idx = PDX(va);
71     uint32_t ptab_idx = PTX(va);
72
73     PDE *pdir = (PDE *)p->ptr;
74     if (!(pdir[pdir_idx] & PTE_P)) {
75         PTE *ptab = (PTE *)palloc_f();
76         memset(ptab, 0, PGSIZE);
77         pdir[pdir_idx] = PTE_ADDR(ptab) | PTE_P | PTE_W | PTE_U;
78     }
79
80     PTE *ptab = (PTE *)PTE_ADDR(pdir[pdir_idx]);
81     ptab[ptab_idx] = PTE_ADDR(pa) | PTE_P | PTE_W | PTE_U;
82 }

```

_map 函数的作用其实就是为某个虚拟页创建对应的页表项，首先通过 PDX(va) 和 PTX(va) 分别计算虚拟地址对应的页目录索引 pdir_idx 和页表索引 ptab_idx。

随后程序通过 p->ptr 获得当前页目录的基地址，并将其转换成 PDE * 类型的 pdir。

接下来函数检查 pdir[pdir_idx] 是否已经存在有效页表，也就是检查页目录项中的 PTE_P 位是否为 1。如果该页目录项不存在，说明当前虚拟地址对应的页表还没有建立，因此程序需要先创建一个新的页表。

创建页表时，函数首先调用 palloc_f() 分配一个新的物理页作为页表，然后使用 memset 将整个页表清零。之后程序将新页表的物理地址写入对应页目录项，同时设置 PTE_P、PTE_W 和 PTE_U 标志位。

完成页表创建后，程序通过页目录项中的地址字段得到对应页表的地址，并将其转换成 PTE * 类型的 ptab。随后根据页表索引 ptab_idx 找到目标页表项，并将物理地址 pa 写入该页表项中，同时设置存在位、可写位和用户权限位。

然后修改 loader() 函数，在 nanos-lite/src/loader.c 里：

```
7  uintptr_t loader(_Protect *as, const char *filename) {
8      if (filename == NULL) {
9          filename = "/bin/hello";
10     }
11
12     int fd = fs_open(filename, 0, 0);
13     assert(fd >= 0);
14
15     size_t size = fs_filesz(fd);
16     uintptr_t entry = (uintptr_t)DEFAULT_ENTRY;
17     uintptr_t start = PGROUNDDOWN(entry);
18     uintptr_t end = PGROUNDUP(entry + size);
19
20     for (uintptr_t va = start; va < end; va += PGSIZE) {
21         void *pa = new_page();
22         memset(pa, 0, PGSIZE);
23         _map(as, (void *)va, pa);
24
25         size_t offset = va < entry ? entry - va : 0;
26         size_t nread = PGSIZE - offset;
27         if (nread > size) {
28             nread = size;
29         }
30
31         if (nread > 0) {
32             fs_read(fd, (uint8_t *)pa + offset, nread);
33             size -= nread;
34         }
35     }
36
37     fs_close(fd);
38     return entry;
39 }
```

整个修改的核心思路就是通过 fs_filesz(fd) 获取文件的总大小，然后设定程序的加载入口地址 entry，随后计算需要加载的虚拟地址范围，其中 start 是入口地址向下对齐到页边界的地址，而 end 是入口地址加文件大小后向上对齐到页边界的地址。

之后从 start 到 end 按页遍历虚拟地址空间，在每一轮循环中，首先通过 new_page() 分配一页新的物理内存，并使用 memset 将其清零，调用 _map(as, va, pa)，将当前虚拟页 va 映射到新分配的物理页 pa。

随后计算当前页需要从文件中读取的数据长度，如果 nread > 0，就调用 fs_read 将文件内容读入到物理页中对应位置。

关闭文件描述符 fs_close(fd)，返回 entry 这个程序的入口地址，CPU 后续会从这个地址开始执行用户程序。

然后运行 dummy 程序，结果如下：

```

SuYaolei@988d7f56373a:~/code/ics2017/nanos-lite$ make run
[src/monitor/monitor.c,65,load_img] The image is /home/SuYaolei/code/ics2017/nanos-lite/build/nanos-lite-x86-nemu.bin
Welcome to NEMU!
[src/monitor/monitor.c,30,welcome] Build time: 19:29:40, May 22 2026
For help, type "help"
(nemu) c
[src/mm.c,44,init_mm] free physical pages starting from 0x1d97000
[src/main.c,20,main] 'Hello World!' from Nanos-lite
[src/main.c,21,main] Build time: 19:29:39, May 22 2026
[src/ramdisk.c,26,init_ramdisk] ramdisk info: start = 0x102bc0, end = 0x1d51805, size = 29682757 bytes
[src/main.c,28,main] Initializing interrupt/exception handler...
nemu: HIT GOOD TRAP at eip = 0x00100032

(nemu) █

```

成功运行。

问题：内核映射的作用

在 `_protect()` 函数中创建虚拟地址空间的时候，有一处代码用于拷贝内核映射：

```

for (int i = 0; i < NR_PDE; i++) {
    updir[i] = kpdirs[i];
}

```

尝试注释这处代码，重新编译并运行，你会看到发生了错误。请解释为什么会发生这个错误。

把这一行注释掉以后，我们编译运行，出现如下的错误：

```

SuYaolei@988d7f56373a:~/code/ics2017/nanos-lite$ make run
(nemu) c
[src/mm.c,44,init_mm] free physical pages starting from 0x1d97000
[src/main.c,20,main] 'Hello World!' from Nanos-lite
[src/main.c,21,main] Build time: 13:56:18, May 24 2026
[src/ramdisk.c,26,init_ramdisk] ramdisk info: start = 0x102c00, end = 0x1d51845, size = 29682757 bytes
[src/main.c,28,main] Initializing interrupt/exception handler...
nemu: src/memory/memory.c:43: page_translate: Assertion `pde.present' failed.
make[1]: *** [Makefile:47: run] Aborted (core dumped)
make[1]: Leaving directory '/home/SuYaolei/code/ics2017/nemu'
make: *** [/home/SuYaolei/code/ics2017/nexus-am/Makefile.app:35: run] Error 2

```

这个循环是在给每个用户进程的页目录补上内核映射，注释掉以后，`_protect()` 只分配了一个空页目录，没有把内核那部分地址空间复制进去，一旦 `_switch()` 切到这个进程的页表，内核代码、内核栈、以及后续 `trap/syscall` 进入内核时要用到的地址就可能不再可访问。

3.在分页机制上运行仙剑奇侠传

之前我们让 `mm_brk()` 函数直接返回 0，表示用户程序的堆区大小修改总是成功，这是因为在实现分页机制之前，0x4000000 之上的内存都可以让用户程序自由使用。现在用户程序运行在虚拟地址空间之上，我们还需要在 `mm_brk()` 中把新申请的堆区映射到虚拟地址空间中，在 `nanos-lite/src/mm.c` 里：

```

18  ∨ int mm_brk(uint32_t new_brk) {
19  ∨     if (current->cur_brk == 0) {
20      |     current->cur_brk = current->max_brk = new_brk;
21      |     }
22  ∨     else {
23  ∨         if (new_brk > current->max_brk) {
24          |         uintptr_t va = PGROUNDUP(current->max_brk);
25          |         uintptr_t end = PGROUNDUP(new_brk);
26          |
27  ∨         for (; va < end; va += PGSIZE) {
28          |             void *pa = new_page();
29          |             memset(pa, 0, PGSIZE);
30          |             _map(&current->as, (void *)va, pa);
31          |         }
32          |
33          |         current->max_brk = new_brk;
34          |     }
35          |
36          |     current->cur_brk = new_brk;
37          |     }
38          |
39          |     return 0;
40      }

```

核心思路是将当前最大堆边界 max_brk 向上页对齐得到 va，同时将新的堆边界 new_brk 向上页对齐得到 end。

随后函数进入循环，从 va 到 end 逐页处理，每次循环中，通过 new_page() 分配一页新的物理内存，并使用 memset 将其清零，调用 _map() 将虚拟地址 va 映射到刚分配的物理页 pa，从而扩展进程的地址空间。

完成所有页的映射后，将 current->max_brk 更新为 new_brk。

然后修改 nanos-lite/src/syscall.c 的 do_syscall 函数：

```

33      |     case SYS_brk:
34      |         SYSCALL_ARG1(r) = mm_brk(a[1]);
35      |         return r;

```

之后尝试运行仙剑奇侠传，结果如下：

```
C irq.c  C exec.c  C proc.c  C intr.c  C asye.c  C reg.h  C monitor.c  trap.S  C syscall.c  X  C nanos.c  C decode.h  C de

nanos-lite > src > C syscall.c
7  _RegSet* do_syscall(_RegSet *r) {
14  switch (a[0]) {
24  case SYS_close:
25      SYSCALL_ARG1(r) = fs_close(a[1]);
26      return r;
27  case SYS_write:
28      SYSCALL_ARG1(r) = fs_write(a[1], (const void *)a[2], a[3]);
29      return r;
30  case SYS_lseek:
31      SYSCALL_ARG1(r) = fs_lseek(a[1], a[2], a[3]);
32      return r;
33  case SYS_brk:
34      SYSCALL_ARG1(r) = mm_brk(a[1]);
35      return r;
36  case SYS_exit:
37      _halt(a[1]);
38      return NULL;
39  default: panic("Unhandled syscall ID = %d", a[0]);
40  }
41
42  return NULL;
43  }
44

问题  输出  调试控制台  终端  窗口

SuYaoLei@988d7f56373a:~/code/ics2017/nanos-lite$ make run
[src/main.c,20,main] 'Hello World!' from Nanos-lite
[src/main.c,21,main] Build time: 19:38:50, May 22 2026
[src/ramdisk.c,26,init_ramdisk] ramdisk info: start = 0x102ba0, end = 0x102ba0
[src/main.c,28,main] Initializing interrupt/exception handler...
game start!
VIDEO_Init success
loading fbp.mkf
loading mgo.mkf
loading ball.mkf
loading data.mkf
loading f.mkf
loading fire.mkf
```

能看到成功运行，实现是正确的。

4.实现内核自陷

根据指导书，首先修改 nanos-lite/src/main.c：

```
37  load_prog("/bin/pal");
38
39  _trap();
40
```

以及 nanos-lite/src/proc.c 的 load_prog() 函数：

```
16  uintptr_t entry = loader(&pcb[i].as, filename);
17
18  // TODO: remove the following three lines after you have implemented _umake()
19  // _switch(&pcb[i].as);
20  // current = &pcb[i];
21  // ((void (*)(void))entry)();
22
```

在这里通过自陷的方式触发第一次上下文切换，而在 x86-nemu 的 AM 中，约定内核自陷通过指令 int \$0x81 触发，因此我们补全 nexus-am/am/arch/x86-nemu/src/asye.c 的 _trap() 函数：

```
53  void _trap() {
54      asm volatile("int $0x81");
55  }
```

并在 irq_handle() 函数中添加相应的的处理：

```

15     switch (tf->irq) {
16     case 0x80: ev.event = _EVENT_SYSCALL; break;
17     case 0x81: ev.event = _EVENT_TRAP; break;

```

并在 nanos-lite/src/irq.c 里补充自陷的处理，让其可以识别内核自陷并包装成 _EVENT_TRAP 事件：

```

8  static _RegSet* do_event(_Event e, _RegSet* r) {
9      switch (e.event) {
10         case _EVENT_SYSCALL:
11             // do_syscall(r);
12             // return schedule(r);
13             return do_syscall(r);
14         case _EVENT_TRAP:
15             printf("Got self-trap event\n");
16             return schedule(r);

```

以及 nexus-am/am/arch/x86-nemu/src/asye.c 的 _asye_init() 函数：

```

33 void _asye_init(_RegSet*(*h)(_Event, _RegSet*)) {
34     // initialize IDT
35     for (unsigned int i = 0; i < NR_IRQ; i++) {
36         idt[i] = GATE(STS_TG32, KSEL(SEG_KCODE), vecnull, DPL_KERN);
37     }
38
39     // ----- system call -----
40     idt[0x80] = GATE(STS_TG32, KSEL(SEG_KCODE), vecsys, DPL_USER);
41     idt[0x81] = GATE(STS_TG32, KSEL(SEG_KCODE), vectrap, DPL_KERN);

```

并在 nexus-am/am/arch/x86-nemu/src/trap.S 里补充：

```

1  #----|-----entry-----|-----errorcode-----|---irq id---|---handler---|
2  .globl vecsys;   vecsys:  pushl $0;  pushl $0x80; jmp asm_trap
3  .globl vecnull; vecnull:  pushl $0;  pushl $-1;  jmp asm_trap
4  .globl vectrap; vectrap:  pushl $0;  pushl $0x81; jmp asm_trap

```

5.实现上下文切换

接下来我们就需要实现 PTE 的 _umake() 函数，在 nexus-am/am/arch/x86-nemu/src/pte.c 里：

```

87  _RegSet *_umake(_Protect *p, _Area ustack, _Area kstack, void *entry, char *const argv[], char *const envp[]) {
88      (void)p;
89      (void)kstack;
90      (void)argv;
91      (void)envp;
92
93      uintptr_t *sp = (uintptr_t *)ustack.end;
94      sp -= 4;
95
96      _RegSet *tf = (_RegSet *)sp - 1;
97      memset(tf, 0, sizeof(*tf));
98
99      sp[0] = 0; // fake return address
100     sp[1] = 0; // argc
101     sp[2] = 0; // argv
102     sp[3] = 0; // envp
103
104     tf->eip = (uintptr_t)entry;
105     tf->cs = 8;
106     tf->eflags = 0x2;
107     tf->esp = (uintptr_t)sp;
108     tf->iqr = 0;
109     tf->error_code = 0;
110
111     return tf;
112 }

```

_umake 函数的作用是为用户进程构造初始运行环境，

首先将用户栈顶地址 ustack.end 强制转换为 uintptr_t *，作为栈指针 sp，并向下预留了 4 个字的空间。

接下来，函数定义了一个 _RegSet *tf，它指向栈中更低的位置，用来保存 CPU 的初始寄存器状态，并通过 memset 将整个 _RegSet 清零。

然后函数开始构造用户栈的初始内容并初始化 _RegSet 结构。

最后返回 tf，这个寄存器结构会被调度器使用，在上下文切换时恢复 CPU 状态，从而让程序从 entry 开始执行。

然后需要编写 nanos-lite/src/proc.c 的 schedule() 函数：

```

30  _RegSet* schedule(_RegSet *prev) {
31      if (current != NULL) {
32          current->tf = prev;
33      }
34      current = &pcb[0];
35      _switch(&current->as);
36      return current->tf;
37  }

```

它用于在两个进程之间进行切换，并完成上下文保存与恢复。

首先保存当前进程的上下文。如果 current 不为空，那么就将传入的 _RegSet *prev 保存到 current->tf 中。

接下来函数进行进程切换，并调用 _switch(¤t->as) 来切换进程的地址空间，更新页表基址。

最后，函数返回 current->tf 作为新进程的寄存器现场，调度器会利用它恢复 CPU 状态，使新的进程从上次保存的位置继续执行。

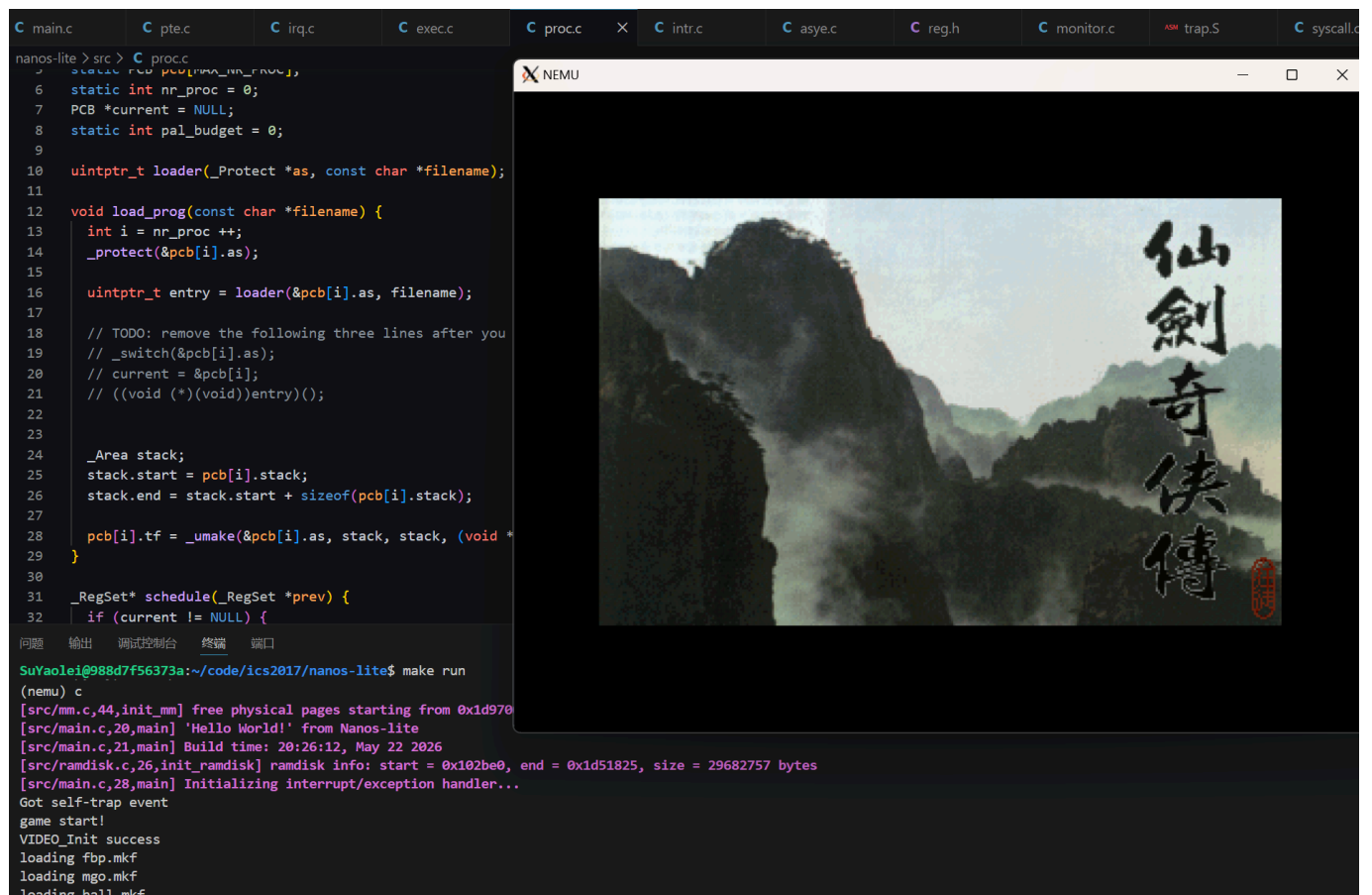
并且修改 nexus-am/am/arch/x86-nemu/src/trap.S：

```

7   asm_trap:
8       pushal
9
10      pushl %esp
11      call irq_handle
12
13      addl $4, %esp
14
15      movl %eax, %esp
16
17      popal
18      addl $8, %esp
19
20      iret

```

然后我们运行仙剑奇侠传，结果如下：



```

C main.c  C pte.c  C irq.c  C exec.c  C proc.c  X  C intr.c  C asye.c  C reg.h  C monitor.c  ASM trap.S  C syscall.c
nanos-lite > src > C proc.c
1 static pcb_t pcb[MAX_PROC];
2 static int nr_proc = 0;
3 PCB *current = NULL;
4 static int pal_budget = 0;
5
6 uintptr_t loader(_Protect *as, const char *filename);
7
8 void load_prog(const char *filename) {
9     int i = nr_proc++;
10    _protect(&pcb[i].as);
11
12    uintptr_t entry = loader(&pcb[i].as, filename);
13
14    // TODO: remove the following three lines after you
15    // _switch(&pcb[i].as);
16    // current = &pcb[i];
17    // ((void (*)(void))entry)();
18
19    _Area stack;
20    stack.start = pcb[i].stack;
21    stack.end = stack.start + sizeof(pcb[i].stack);
22
23    pcb[i].tf = _umake(&pcb[i].as, stack, stack, (void *)
24 }
25
26 _RegSet* schedule(_RegSet *prev) {
27     if (current != NULL) {

```

```

SuYaoLei@988d7f56373a:~/code/ics2017/nanos-lite$ make run
(nemu) c
[src/mm.c,44,init_mm] free physical pages starting from 0xd970
[src/main.c,20,main] 'Hello World!' from Nanos-lite
[src/main.c,21,main] Build time: 20:26:12, May 22 2026
[src/ramdisk.c,26,init_ramdisk] ramdisk info: start = 0x102be0, end = 0xd51825, size = 29682757 bytes
[src/main.c,28,main] Initializing interrupt/exception handler...
Got self-trap event
game start!
VIDEO_Init success
loading fbp.mkf
loading mgo.mkf
loading hall.mkf

```

能看到输出 "Got self-trap event" 说明 Nanos-lite 可以通过内核自陷触发上下文切换的方式运行仙剑奇侠传了。

6.分时运行仙剑奇侠传和hello程序

在 nanos-lite/src/main.c 里补充：

```

37     load_prog("/bin/pal");
38     load_prog("/bin/hello");
39
40     _trap();

```

之后，需要修改调度的代码，让 schedule() 轮流返回仙剑奇侠传和 hello 的现场：

```

31 _RegSet* schedule(_RegSet *prev) {
32     if (current != NULL) {
33         current->tf = prev;
34     }
35     current = (current == &pcb[0] ? &pcb[1] : &pcb[0]);
36     _switch(&current->as);
37     return current->tf;
38 }

```

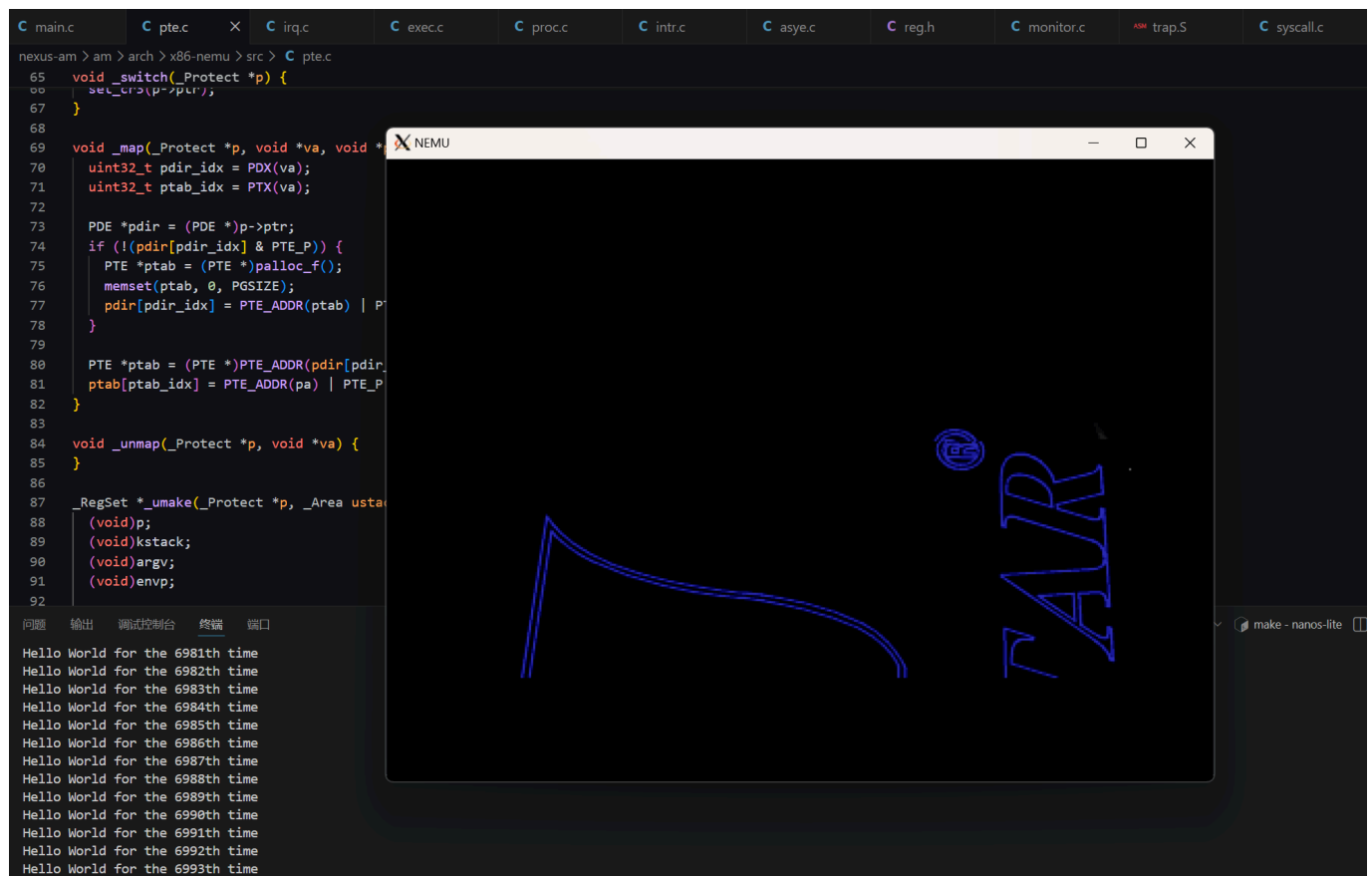
并修改 nanos-lite/src/irq.c 里 do_event() 的代码，在处理完系统调用之后，调用 schedule() 函数并返回其现场：

```

8 static _RegSet* do_event(_Event e, _RegSet* r) {
9     switch (e.event) {
10         case _EVENT_SYSCALL:
11             do_syscall(r);
12             return schedule(r);
13             // return do_syscall(r);
14         case _EVENT_TRAP:
15             printf("Got self-trap event\n");
16             return schedule(r);

```

然后我们就可以运行仙剑奇侠传和 hello 程序了，结果如下：



The screenshot shows the NEMU emulator interface. On the left, a terminal window displays the output of the 'hello' program, which prints 'Hello World for the 6981th time' through 'Hello World for the 6993th time'. On the right, a window titled 'NEMU' shows a graphical representation of the '仙剑奇侠传' (Sword and Fairy) program, featuring a blue outline of a sword and the characters '仙剑' in a stylized font. The top of the interface shows a list of loaded modules: main.c, pte.c, irq.c, exec.c, proc.c, intr.c, asy.c, reg.h, monitor.c, trap.S, and syscall.c.

能看到仙剑奇侠传一边运行的同时，hello 程序也会一边输出，实现是正确的。

7. 优先级调度

我们可以修改 schedule() 的代码，来调整仙剑奇侠传和 hello 程序调度的频率比例，使得仙剑奇侠传调度若干次，才让 hello 程序调度 1 次，

```

8  static int pal_budget = 0;

31 _RegSet* schedule(_RegSet *prev) {
32     if (current != NULL) {
33         current->tf = prev;
34     }
35     // current = (current == &pcb[0] ? &pcb[1] : &pcb[0]);
36
37     if (current == NULL) {
38         current = &pcb[0];
39         pal_budget = 0;
40     }
41     else if (current == &pcb[0]) {
42         if (nr_proc > 1 && pal_budget + 1 >= 5000) {
43             current = &pcb[1];
44             pal_budget = 0;
45         }
46         else {
47             current = &pcb[0];
48             pal_budget ++;
49         }
50     }
51     else {
52         current = &pcb[0];
53     }
54
55     _switch(&current->as);
56     return current->tf;
57 }

```

和之前的有区别的是先判断 `current` 是否为空，如果为空说明是系统第一次调度，此时直接选择 `pcb[0]` 作为第一个运行的进程，并将 `pal_budget` 清零，表示从头开始计算调度预算。这里 `pcb[0]` 就是仙剑奇侠传，而 `pcb[1]` 就是 hello 程序。

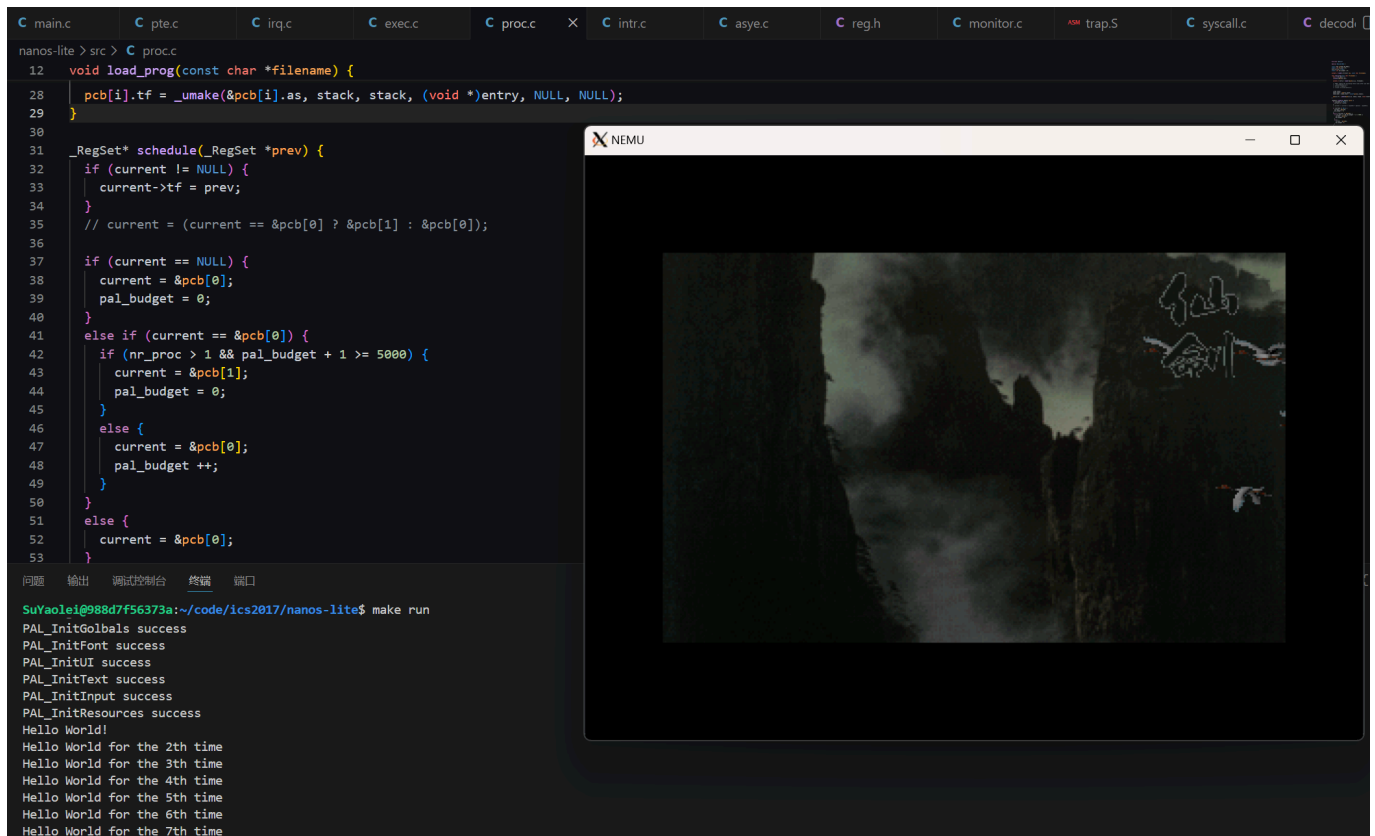
如果当前运行的是 `pcb[0]`，函数会进一步判断是否应该切换到 `pcb[1]`，这里的条件是 `nr_proc > 1` 并且 `pal_budget + 1 >= 5000`，也就是 `pcb[0]` 的连续运行次数已经达到限制，需要让出 CPU。

如果满足条件，就将 `current` 切换到 `pcb[1]`，并把 `pal_budget` 重置为 0，表示重新开始计算另一进程的运行预算。

如果条件不满足，则仍然保持运行 `pcb[0]`，同时将 `pal_budget` 自增 1，表示 `pcb[0]` 继续占用 CPU 时间片。

如果当前运行的是 `pcb[1]`，那么函数会直接切换回 `pcb[0]`。

运行结果如下：



这次仙剑奇侠传运行的速度就比上次要快很多，并且仙剑奇侠传一边运行的同时，hello 程序也会一边输出，实现正确。

8.添加时钟中断

根据指导书，需要在 cpu 结构体中添加一个 bool 成员 INTR，在 nemu/include/cpu/reg.h 里：

```
60 CR0 cr0;
61 CR3 cr3;
62
63 bool INTR;
```

然后在 nemu/src/cpu/intr.c 的 dev_raise_intr() 中将 INTR 引脚设置为高电平：

```
24 void dev_raise_intr() {
25     cpu.INTR = true;
26 }
```

并在 nemu/src/cpu/exec/exec.c 的 exec_wrapper() 的末尾添加轮询 INTR 引脚的代码，每次执行完一条指令就查看是否有硬件中断到来：

```
15 #define TIMER_IRQ 32

253 update_eip();
254
255 if (cpu.INTR & cpu.IF) {
256     cpu.INTR = false;
257     raise_intr(TIMER_IRQ, cpu.eip);
258 }
```

然后我们修改 nemu/src/cpu/intr.c 里 raise_intr() 中的代码，在保存 EFLAGS 寄存器后，将其 IF 位置为 0，让处理器进入关中断状态：

```
4 void raise_intr(uint8_t NO, vaddr_t ret_addr) {
5     /* TODO: Trigger an interrupt/exception with ``NO``.
6      * That is, use ``NO`` to index the IDT.
7      */
8
9     rtlreg_t idt_base = cpu.idtr.base;
10    rtlreg_t desc_lo = vaddr_read(idt_base + NO * 8, 4);
11    rtlreg_t desc_hi = vaddr_read(idt_base + NO * 8 + 4, 4);
12    vaddr_t target = (desc_hi & 0xffff0000) | (desc_lo & 0xffff);
13
14    rtl_push(&cpu.eflags);
15    cpu.IF = 0;
16    rtl_li(&t0, 0x8);
17    rtl_push(&t0);
18    rtl_li(&t0, ret_addr);
19    rtl_push(&t0);
20
21    cpu.eip = target;
22 }
```

在软件上，你还需要在 nexus-am/am/arch/x86-nemu/src/asye.c 中添加时钟中断的支持，将时钟中断打包成 _EVENT_IRQ_TIME 事件：

```
11 _RegSet* irq_handle(_RegSet *tf) {
12     _RegSet *next = tf;
13     if (H) {
14         _Event ev;
15         switch (tf->irq) {
16             case 0x80: ev.event = _EVENT_SYSCALL; break;
17             case 0x81: ev.event = _EVENT_TRAP; break;
18             case 32: ev.event = _EVENT_IRQ_TIME; break;
19             default: ev.event = _EVENT_ERROR; break;
20         }
21
22         next = H(ev, tf);
23         if (next == NULL) {
24             next = tf;
25         }
26     }
27
28     return next;
29 }
```

```

33 void _asye_init(_RegSet*(*h)(_Event, _RegSet*)) {
34     // initialize IDT
35     for (unsigned int i = 0; i < NR_IRQ; i++) {
36         idt[i] = GATE(STS_TG32, KSEL(SEG_KCODE), vecnull, DPL_KERN);
37     }
38
39     // ----- system call -----
40     idt[0x80] = GATE(STS_TG32, KSEL(SEG_KCODE), vecsys, DPL_USER);
41     idt[0x81] = GATE(STS_TG32, KSEL(SEG_KCODE), vectrap, DPL_KERN);
42     idt[32] = GATE(STS_IG32, KSEL(SEG_KCODE), vectime, DPL_KERN);
43     set_idt(idt, sizeof(idt));
44
45     // register event handler
46     H = h;
47 }

```

同样的，补充 nexus-am/am/arch/x86-nemu/src/trap.S:

```

1  #----|-----entry-----|-----errorcode|---irq id---|---handler---|
2  .globl vecsys;    vecsys:  pushl $0;  pushl $0x80; jmp asm_trap
3  .globl vecnull;   vecnull: pushl $0;  pushl $-1;  jmp asm_trap
4  .globl vectrap;   vectrap: pushl $0;  pushl $0x81; jmp asm_trap
5  .globl vectime;   vectime: pushl $0;  pushl $32;  jmp asm_trap

```

然后，Nanos-lite 收到 `_EVENT_IRQ_TIME` 事件之后，直接调用 `schedule()` 进行进程调度，同时也可以去掉系统调用之后调用的 `schedule()` 代码了，在 `nanos-lite/src/irq.c` 的 `do_event()` 里：

```

8  static _RegSet* do_event(_Event e, _RegSet* r) {
9      switch (e.event) {
10         case _EVENT_SYSCALL:
11             // do_syscall(r);
12             // return schedule(r);
13             return do_syscall(r);
14         case _EVENT_TRAP:
15             printf("Got self-trap event\n");
16             return schedule(r);
17         case _EVENT_IRQ_TIME:
18             printf("timer interrupt\n");
19             return schedule(r);
20         default: panic("Unhandled event ID = %d", e.event);
21     }
22
23     return NULL;
24 }

```

而为了可以让处理器在运行用户进程的时候响应时钟中断，还需要修改 `nexus-am/am/arch/x86-nemu/src/pte.c` 里 `_umake()` 的代码，在构造现场的时候，设置正确的 `EFLAGS`，把 `EFLAGS` 的第 1 位和 `IF` 位都置 1，也就是 `0x202`：

```

104     tf->eip = (uintptr_t)entry;
105     tf->cs = 8;
106     tf->eflags = 0x202;
107     tf->esp = (uintptr_t)sp;
108     tf->irq = 0;
109     tf->error_code = 0;

```

然后我们就可以运行仙剑奇侠传和 hello 程序了，结果如下：

The screenshot shows the Nanos-lite development environment. The top bar lists several source files: main.c, pte.c, irq.c, exec.c, cpu-exec.c, proc.c, intr.c, asye.c, reg.h, monitor.c, and trap.S. The main window displays the source code of proc.c, which includes functions for loading programs and scheduling. The bottom terminal window shows the output of the 'hello' program, which prints 'Hello World for the 573th time' through 'Hello World for the 576th time', followed by several 'timer interrupt' messages. To the right, a window titled 'X NEMU' displays a screenshot of the 'Xianjian' (仙剑) game, showing a dark, atmospheric scene with the title '仙剑' in the top right corner.

能看到输出“timer interrupt”，说明 Nanos-lite 是在时钟中断的时候来进行仙剑奇侠传和 hello 两个进程的切换的，实现正确。

问题：灾难性的后果

假设硬件把中断信息固定保存在内存地址 0x1000 的位置, AM 也总是从这里开始构造 trap frame. 如果发生了中断嵌套, 将会发生什么样的灾难性后果? 这一灾难性的后果将会以什么样的形式表现出来? 如果你觉得毫无头绪, 你可以用纸笔模拟中断处理的过程。

如果硬件总是把中断现场固定保存到内存地址 0x1000，并且 AM 在处理中断时也始终从这个位置构造 trap frame，那么在发生中断嵌套时，后一次中断会覆盖前一次中断保存的现场信息，导致之前的执行状态彻底丢失。

正常情况下，每一次中断都应该拥有独立的 trap frame，这样即使发生中断嵌套，系统也能够像栈一样一层层保存和恢复不同中断对应的执行状态。但如果硬件规定所有中断都固定写入 0x1000，问题就会出现。

假设程序 A 正在执行，此时发生第一次中断。CPU 会把程序 A 的寄存器和返回地址保存到 0x1000，然后开始执行中断处理函数。但是在第一个中断处理过程中，如果又发生了第二次中断，那么 CPU 会再次把当前现场写

入 0x1000。由于地址固定，因此第二次中断保存的信息会直接覆盖第一次中断原本保存的数据。这样一来，第一个中断对应的 trap frame 就被破坏了。

接下来，当第二个中断处理结束时，系统会根据 0x1000 中的数据恢复现场，此时恢复的是第二个中断发生时的状态，因此这一层可能暂时还能正常返回。但当第一个中断也准备返回时，它仍然会尝试从 0x1000 恢复自己的现场。然而此时 0x1000 中保存的早已不是第一次中断的数据，而是已经被第二次中断覆盖后的内容。

于是 CPU 会恢复出错误的寄存器值、错误的栈指针、错误的程序计数器，于是可能会发生返回到错误的代码地址执行、栈指针损坏，导致函数调用链崩溃、通用寄存器内容错乱、程序跳转到随机地址、覆盖内核数据结构等等问题，最终导致系统崩溃。

9.展示你的计算机系统

首先在 nanos-lite/src/main.c 里我们补充如下内容，添加 videotest：

```
37     load_prog("/bin/pal");
38     load_prog("/bin/hello");
39     load_prog("/bin/videotest");
40
41     _trap();
```

然后我们在 nanos-lite/src/proc.c 里添加一个辅助函数用来切换进程：

```
32     void switch_game(void) {
33         if (current_game == NULL) {
34             current_game = &pcb[0];
35             return;
36         }
37
38         current_game = (current_game == &pcb[0] ? &pcb[2] : &pcb[0]);
39         game_budget = 0;
40     }
```

并修改 schedule()，使用 current_game 来表示当前的前台游戏程序：

```

72 _RegSet* schedule(_RegSet *prev) {
73     if (current != NULL) {
74         current->tf = prev;
75     }
76
77     if (current_game == NULL) {
78         current_game = &pcb[0];
79     }
80
81     if (current == NULL) {
82         current = current_game;
83     }
84     else if (current == current_game) {
85         if (nr_proc > 1 && game_budget + 1 >= 10) {
86             current = &pcb[1];
87             game_budget = 0;
88         }
89         else {
90             current = current_game;
91             game_budget ++;
92         }
93     }
94     else if (current == &pcb[1]) {
95         current = current_game;
96     }
97     else {
98         current = current_game;
99     }
100
101     _switch(&current->as);
102     return current->tf;
103 }

```

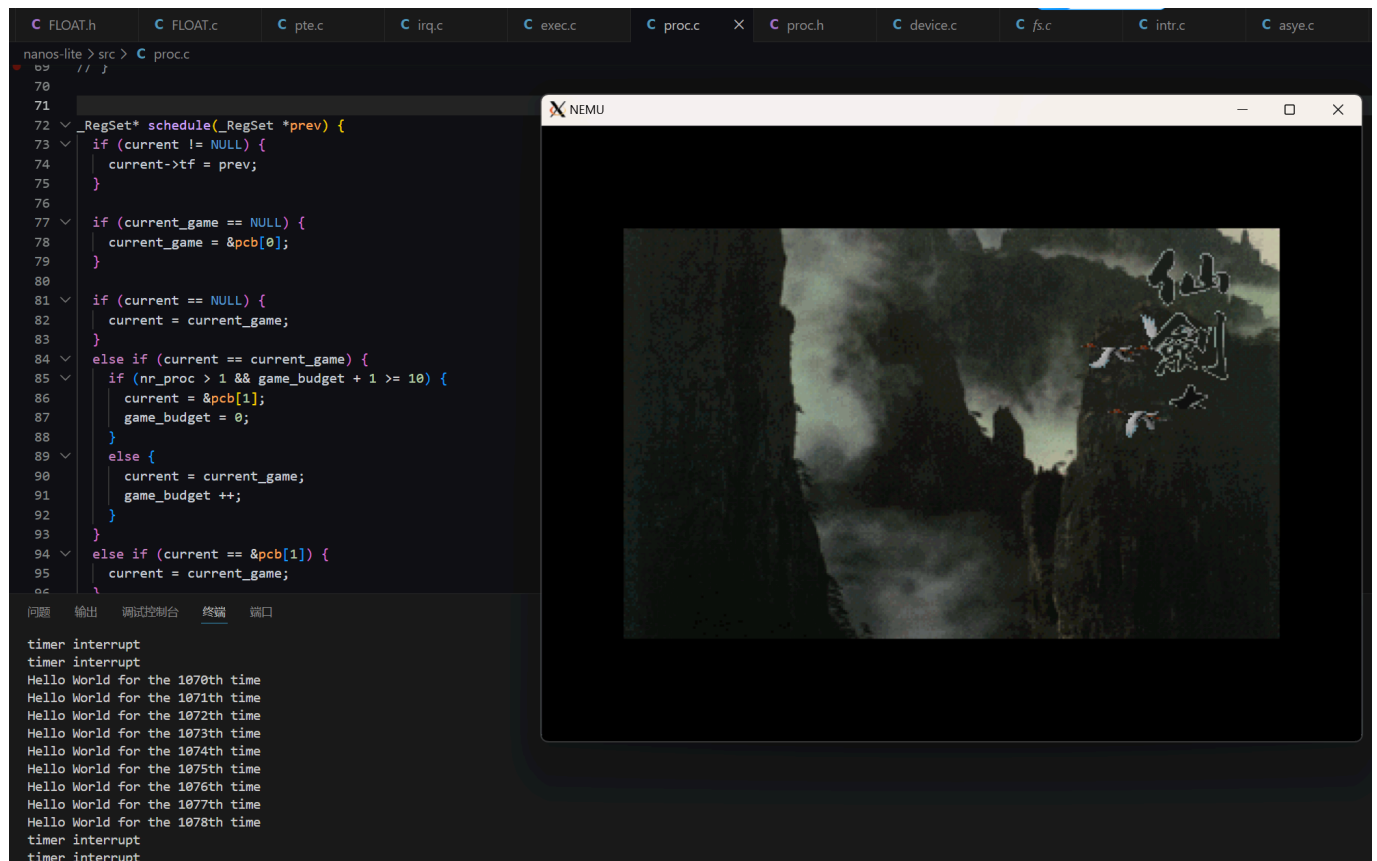
然后我们修改 nanos-lite/src/device.c 的 events_read() 函数，在按下 F12 的时候，让游戏在仙剑奇侠传和 videotest 之间切换：

```

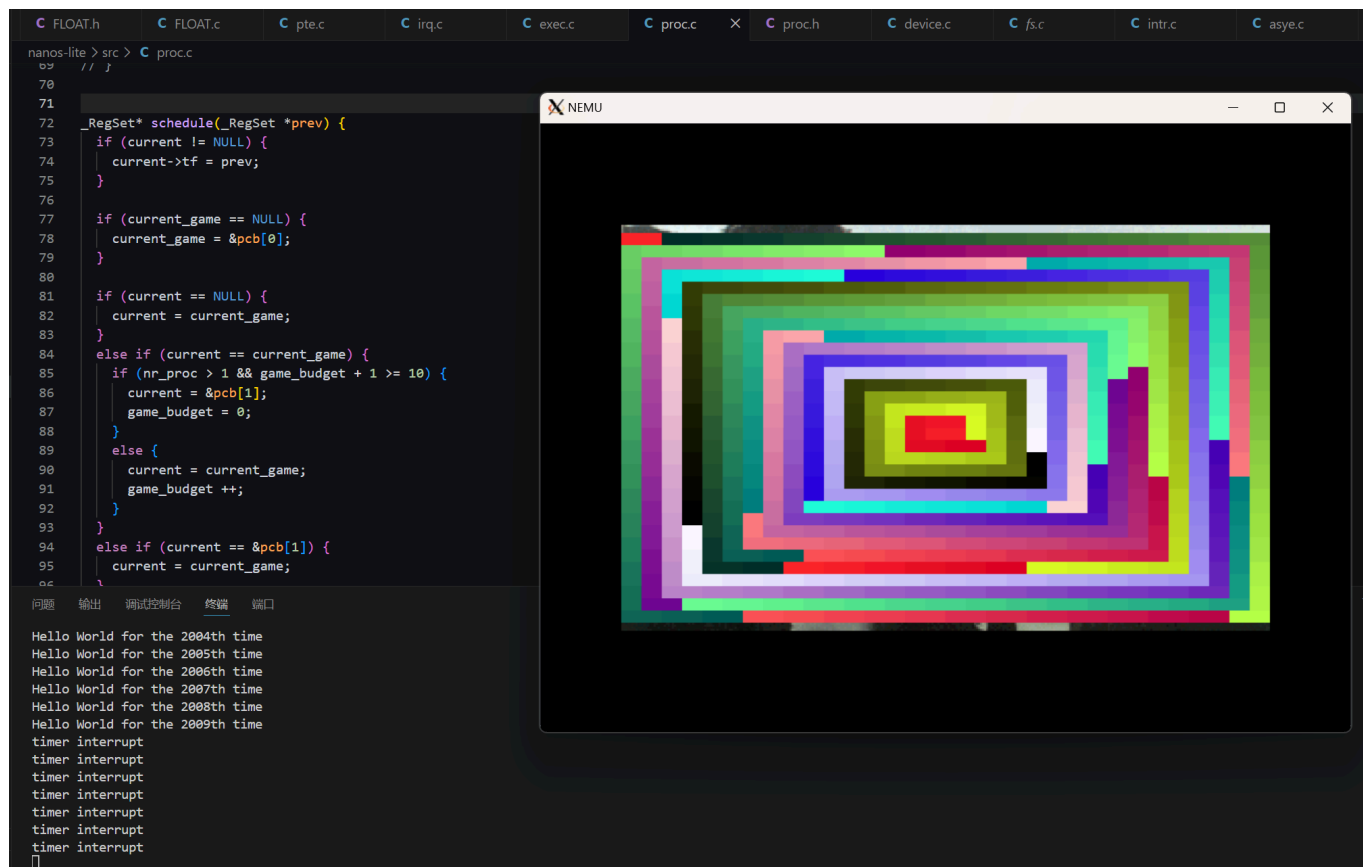
14 size_t events_read(void *buf, size_t len) {
15     char event[32];
16     int key = _read_key();
17
18     if (key != _KEY_NONE) {
19         // const char *type = (key & KEYDOWN_MASK) ? "kd" : "ku";
20
21         int keydown = key & KEYDOWN_MASK;
22         key &= ~KEYDOWN_MASK;
23         if (keydown && key == _KEY_F12) {
24             switch_game();
25         }
26         const char *type = keydown ? "kd" : "ku";
27
28         sprintf(event, "%s %s\n", type, keyname[key]);
29     }
30     else {
31         sprintf(event, "t %u\n", (unsigned)_uptime());
32     }
33
34     size_t event_len = strlen(event);
35     if (len > event_len) {
36         len = event_len;
37     }
38     memcpy(buf, event, len);
39     return len;
40 }

```

然后我们运行程序，得到结果如下：



按下 F12:



能看到，在 hello 程序执行的同时，我们完成了仙剑奇侠传和 videotest 的切换，实现是正确的。

四、必答题

请结合代码, 解释分页机制和硬件中断是如何支撑仙剑奇侠传和hello程序在我们的计算机系统(Nanos-lite, AM, NEMU)中分时运行的.

在我们的计算机系统中，分页机制负责为不同程序提供独立的地址空间，而硬件中断负责周期性打断当前程序并触发调度，两者结合，操作系统才能不断在不同进程之间切换，使多个程序看起来像是在同时运行。

系统启动时 `_pte_init` 会建立内核页表，而 `_protect` 在创建用户进程时，会为每个进程分配新的页目录，并把内核空间对应的页目录项复制到新的页表中，使得每个进程都拥有自己独立的用户态地址空间，因此 hello、仙剑奇侠传等程序之间的数据彼此隔离，一个程序无法直接访问另一个程序的私有内存。

与此同时，所有进程的页表中又都包含相同的内核映射，当发生系统调用、中断或者异常时，CPU 立即进入内核执行。

随后，`_map` 会负责建立虚拟地址到物理地址之间的映射关系，每个用户程序拥有自己的物理页，这样，hello 和仙剑奇侠传即使都认为自己占用了“同一个地址”，实际上访问的也是不同的物理页。

在进程切换时，系统并不会复制整个内存空间，而只是调用 `_switch` 修改 CR3，CR3 中保存的是当前页目录的物理地址，因此当 CR3 被切换后，CPU 使用的页表也随之改变。由于分页机制决定了整个虚拟地址空间的映射关系，所以 CR3 一改变，CPU 看到的内存世界也会立刻变化。

之后 `_umake` 则负责为进程伪造初始执行现场，在进程第一次运行之前，系统会提前构造好 trap frame，包括设置 eip 指向程序入口、esp 指向用户栈顶，并且设置 eflags 的值。

在 NEMU 中，设备触发时钟中断时，会通过 `dev_raise_intr` 设置 `cpu.INTR`。CPU 在执行循环中会不断检查是否存在中断请求，如果发现 `INTR=1` 且当前 `IF` 位允许中断，那么 CPU 就会调用 `raise_intr` 进入中断处理流程。

`raise_intr` 的工作是保存当前执行现场，把当前的 `eflags`、`cs` 和 `eip` 压入栈中，并清除 `IF` 位，防止处理中断时再次被打断，这样，当前程序的执行状态就被冻结在一个可恢复的位置，以后调度回来时，CPU 可以利用这些保存的信息继续执行原来的程序。

接下来，AM 会接管中断处理流程。在 `asye.c` 中，系统会建立 IDT，其中 32 号中断对应时钟中断。时钟中断发生后，`trap.S` 里会先压入 IRQ 编号，再进入 `asm_trap` 保存通用寄存器，之后 `irq_handle` 会把底层 IRQ 号翻译成统一事件 `_EVENT_IRQ_TIME`。

随后在 Nanos-lite 中，`irq.c` 收到 `_EVENT_IRQ_TIME` 后，就会调用 `schedule`。调度器会先把当前进程的 `trap frame` 保存到 PCB 中，然后选择下一个进程运行。

在 `proc.c` 中，系统会为 `hello`、仙剑奇侠传等程序分别创建 PCB，每个 PCB 都保存自己的 `trap frame` 和页表。调度器每次切换进程时，都会恢复不同 PCB 中保存的上下文，并切换对应页表。因此 `hello` 和仙剑奇侠传虽然实际上是轮流占用 CPU，但由于每次恢复时都能回到自己之前暂停的位置，所以从用户角度看，它们就像是在同时运行。

因此，整个系统中的分时运行本质上是如下的过程：分页机制为每个程序提供独立地址空间，时钟中断周期性打断当前程序，AM 将硬件中断转换成统一事件，而 Nanos-lite 的调度器则负责保存当前现场、切换页表并恢复下一个进程。CPU 实际上在不同程序之间高速切换，而由于切换速度极快，人就会感觉仙剑奇侠传和 `hello` 像是在同时运行。

五、实验过程中的问题与解决方法

在实现调度器切换的时候，调度器和装载顺序设置反了，先加载的程序是 `pcb[0]`，如果先加载 `hello` 程序，就会出现 `hello` 程序被调度几十几百次，才会调度一次仙剑奇侠传，然后我们的游戏就会加载地非常缓慢。

在实现时钟中断时，函数 `exec_wrapper()` 里，我在时钟中断触发后先执行了 `raise_intr(TIMER_IRQ, cpu.eip);`，但后面又立刻执行了一次 `update_eip();`，然后就报错了，究其原因，是把刚刚写进去的中断入口地址覆盖回顺序执行地址了，这样不会跳进中断处理流程，所以把 `update_eip();` 删掉即可。

以及在实现多个进程切换时，必须确保装载了相应数量的程序，否则会访问未初始化的 PCB，程序执行发生错误，总体来说，PA4 还是让我收获颇丰。